

The World Wide Web

The *internet* refers to the actual network of connected computing devices. Although the internet was around in the 1980s, there was not an intuitive way for most people to browse the internet. The internet just sent messages produced by one computer and presented them to another computer.

Browsers and Servers

One way of understanding this process is to look at the *client-server* model. In this model, the *client* refers to the user's device or program that is making a request for data. A client can be a browser or application running on a user's laptop, smartphone, or tablet.

The *server* is the device or program in that network that waits for incoming requests and sends back data. This might be an in-house server, a rented server at a data center, or cloud server. At Codecademy, we have servers that store lesson data and our servers are sending this lesson data to your client device.

404 Status Code

When a server responds to a client, the server specifies a status code as a part of the response. Status codes indicate whether or not the HTTP request was successfully completed and if there was an error, they contain some information about the type of error that happened. The status code helps the browser know how to handle the data that was sent back with the response.

Review the HTTP statuses below and see if any of them seem familiar.

Code	Explanation
200 OK	The request has succeeded.
301 Moved Permanently	The resource has been moved and the client is being redirected.
404 Not Found	The requested resource was not found.
500 Internal Server Error	The server encountered an unexpected error.

How Do Browsers Work?

All of the following steps happen in a split second:

1. When a user types in a URL and presses enter, the server processes the request and sends the HTML file back to the client. *HTML* files hold the content of a website and they also contain links for any additional assets or code that are needed to display the site properly.

2. The browser will begin to search for elements in the HTML file and it will start to make additional HTTP requests for any other external resources used by the HTML file. This often includes:

=> One or more CSS stylesheets. *CSS* stands for cascading style sheets; *CSS* creates the style and layout of a web page. The browser will request the *CSS* stylesheet, and when the server sends it back, the browser analyzes the *CSS* and starts applying the visual styles to the content of the site.

=> The request-response cycle also sends website assets, like images and videos, from the server to the browser. If these files are large, there might even be a noticeable delay before they are rendered by the browser.

=> One or more JavaScript files. *JavaScript* makes the webpage interactive. This programming language functions as the “behavior” of the web page. A webpage that does not use JavaScript is known as a *static* webpage.

In most modern browsers, these additional requests are made in parallel. This means that these requests are initiated at the same time, and the browser does not wait to receive one resource before requesting the next resource.

All of the resources are typically displayed as soon as possible. The HTML will be displayed even if some of the other assets have not been received by the browser.

Web 2.0

The earliest static websites were composed of text, images, and links, with very little interactivity beyond browsing from one page to another. These websites are called *static*, which means lacking in movement because they do not change based on user behavior. As internet connection speeds and web technologies progressed, more complex interactions became possible on the web.

A collection of advances in the early 2000s created a cluster of web applications that are often called “Web 2.0”. In comparison to early static websites, Web 2.0 applications are often defined by:

=> Providing a dynamic user experience by offering content that responds to user input without forcing the page to reload. In the early web, user input would typically take the user to a new page — and they would have to wait for the new page to load. In Web 2.0, websites could just update selected regions of the page, avoiding the interruption caused by reloading.

=>Emphasizing user-generated content and social sharing. In the early web, content was generally authored by a single source. The rise of blogging, social media, and wikis in web 2.0 meant that users could generate content and share it with their friends.

There were important technical advances that enabled each of these advances in the user interface of the internet. For example:

=> *jQuery* was the first JavaScript framework that could fetch data while the web page is running.

=> The rise of *web frameworks* that connected to databases, like Spring, Django, and Ruby-on-Rails, enabled user-generated content to effectively be created, stored, and displayed.

Web 1.0

Hitsville, USA

"Hitsville U.S.A." is the nickname given to Motown's first headquarters. A former photographers' studio located at 2648 West Grand Boulevard in Detroit, Michigan, near the New Center area, it was purchased by Motown founder [Berry Gordy](#) in 1959.

It was converted for use as the record label's administrative building and recording studio, which was open 22 hours a day (closing from 8 to 10 a.m. for maintenance). Following mainstream success in the late 1960s and early 1970s, Gordy moved the label to Los Angeles and established the Hitsville West studio there, as a part of his focus on television and film production as well as music production.



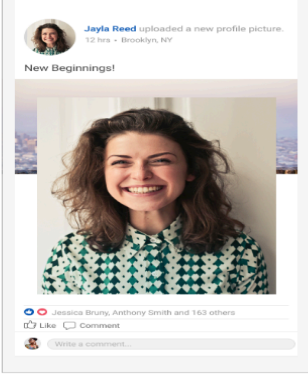
The Hitsville U.S.A. street in Detroit, Michigan.

West Grand Boulevard

In 1959, Gordy formed his first label, Tamla Records, and purchased the property that would become Motown's Hitsville U.S.A. studio. The photography studio located in the back of the property was modified into a small recording studio, and the Gordys moved into the second-floor living quarters. Within seven years, Motown would occupy seven additional neighboring houses:

- Hitsville U.S.A., 1959: (ground floor) administrative office, tape library, control room, Studio A; (upper floor) Gordy living quarters (1959–1962), artists and repertoire (1962–1972)
- Jobete Publishing office, 1961: sales, billing, collections, shipping, and public relations
- Berry Gordy Jr. Enterprises, 1962: offices for Berry Gordy, Jr. and his sister Esther Gordy Edwards
- Finance department, 1965: royalties and payroll
- Artist personal development, 1966: Harvey Fuqua (head of artist development and producer of stage performances), Maxine Powell (instructor in grooming, poise, and social graces for Motown artists), Maurice King (vocal coach, musical director and arranger), Cholly Atkins (house choreography), and rehearsal studios
- Two houses for administrative offices, 1966: sales and marketing, traveling and traffic, and mixing and mastering

Web 2.0



Jayla Reed uploaded a new profile picture.
12 hrs • Brooklyn, NY

New Beginnings!

Jessica Brum, Anthony Smith and 163 others

Like Comment

Write a comment

Current Internet Trends

The rise of internet-connected smartphones has profoundly changed how users interact with the internet. Mobile internet traffic now accounts for more than half of all internet traffic and web development practices have evolved in order to provide a good user experience regardless of device type.

Responsive Web Design

The rise of *responsive web design* has changed how websites are built.

Responsive web design was enabled by additions to the CSS language, like media queries and relative units. These additions allow the presentation of websites to adjust based on the size of the window in which they are displayed.

Mobile Applications and Devices

The rise of internet-connected *mobile applications* has changed the way that we think about browsing the internet. Users accessing the internet on smartphones are likely to spend much more time with specific applications, rather than using their phone's browser.

Though most mobile applications are internet connected, they are not part of the world wide web. The web is built out of links, whereas mobile applications are designed to keep the user's attention.

If you want to learn about mobile development, web development is a great place to start! While the majority of mobile applications are built in programming languages, like Swift for iOS, it is increasingly common to see developers using JavaScript frameworks to build new apps.

What is HTML

HTML is the skeleton of all web pages. It provides structure to the content on a website, including text, images, buttons, videos, and more.

HTML is a practical place to start learning to code. You can build basic websites after learning just a little HTML, with text, images, and videos. You can always open up your work-in-progress website with your browser and see what you're building.

HTML stands for **hypertext markup language**. In the next few exercises, we will look at the HTML acronym word-by-word to unpack what HTML really means.

HTML Markup

The *ML* in HTML stands for *markup language*. Markup refers to a way of annotating text that is distinguishable from the text itself. The same way that a

teacher might “markup” a student essay by underlining topic sentences and circling spelling errors, HTML annotates the content within a web page.

A teacher might use a red pen to make sure that their comments are easy to distinguish from the student’s own work. HTML separates content and annotation by using HTML tags, which are denoted by angle brackets (also known as less-than and greater-than signs).

Comparisons:

=> Both the HTML markup and the essay have sections of text. The HTML markup is separated by new lines and tags with `<>`. The essay has the text separated by new lines.

Both documents include a title near the top. The HTML markup is titled “HTML Tutorial,” marked with `<title>` tags. The essay has “My Autobiography” as the title, which is annotated as “Title” with an arrow.

Similar to the essay’s body text, the HTML has the tag `<body>` for the body text.

In the essay, portions of the body text are annotated to indicate what purpose they serve in the text, such as “Hook,” “Thesis Statement,” and “Conclusion.” In the `<body>` of the HTML, individual portions are annotated with HTML tags like `<p>` and `<h1>` to indicate the function that the text will serve on the web page.

We’ll learn more about these tags in subsequent exercises!

HTML Elements

HTML tags are the “markup” for HTML. They are annotations that provide information about the type of content they contain. Let’s take a close look at the syntax for how HTML tags surround content to create an HTML element.

The diagram displays an HTML paragraph element. As we can see, the paragraph element is made up of one opening tag (`<p>`), the content ("Hello World!" text), and a closing tag (`</p>`).

Let's quickly review each part of the element pictured:

=> HTML element — a unit of content in an HTML document formed by HTML tags and the text or media it contains.

=> Opening Tag — the element name used to start an HTML element. The tag type is surrounded by opening and closing angle brackets.

=> Content — The information (text or other elements) contained between the opening and closing tags of an HTML element.

=> Closing tag — the second HTML tag used to end an HTML element. Closing tags have a forward slash (/) inside of them, directly after the left angle bracket.

Common Tags

Okay, let's practice using HTML elements by writing a few different tags.

Here are a few common tags that you can start getting familiar with:

```
<h1>This is a heading, it emphasizes text.</h1>

<p>This is a paragraph, it is the most common tag for larger
chunks of text.</p>

<a>This is an anchor tag, used to specify the text that is the
"anchor" for a link.</a>

<button>This is a button.</button>
```

Instructions

1.Let's make the heading stand out! Put an opening and closing `h1` tag around the text that says `Jetsetter Travel Agency`.

2.Put an opening and closing `p` tag around the long paragraph that starts with `With over 25 years....`

3.Put an opening and closing `button` tag around the text that says `Click here!`.

```
<h1>Jetsetter Travel Agency</h1>

<p>With over 25 years of experience in concierge, high-end
travel planning, we'll provide you with the highest quality
services. Every vacation is unique, custom, and tailored to your
tastes.</p>

<button>Click here!</button>
```

Hypertext and the World Wide Web

The H and T in HTML stands for **hypertext**. Hypertext is text that is linked to other text. This diagram shows different websites that are connected to each other through links, which are represented by arrows.

What's so hyper about hypertext? The prefix *hyper* indicates that the text expands beyond the traditional constraints of the written word. Instead of reading documents from beginning to end, like you would read a book, someone going through hypertext can *browse*. By clicking on links from one document to another, the user can navigate to whatever page they find the most interesting and carve their own unique path through the web.

Many modern websites provide rich user experiences, with features like embedded videos, animations, and interactivity, so it doesn't necessarily feel like you are just navigating from one HTML page to the next. But despite all of the advances that have taken place with the growth of the web, at its core the web is still just a collection of hyperlinked documents.

Adding Hyperlinks

An *attribute* in HTML provides additional information about an HTML element. It comes in a name and value pair with the structure `name="value"`. For example, you can specify the width of an HTML element with the attribute `width="500"`.

Links are created in HTML with something called the `href` attribute, which stands for **hyperlink reference**. The `href` attribute allows us to specify the URL, or address on the web, that a link should take users to. See below for an example of the `href` attribute in action.

When we set the `href` attribute on an anchor tag (abbreviated to `<a>`) we can specify both the text that should be rendered for the user (the text within the anchor tag) and the URL that the browser should navigate to.

With this code, we're assigning the value `www.codecademy.com` to the `href` attribute. When a user clicks on the text of this link (Learn to code!), they will be directed to `www.codecademy.com`.

```
<a href="www.codecademy.com">Learn to code!</a>
```

Instructions

1. Alejandra has begun to build out her business's website. She wants to link from her homepage to a different page that contains her resume. To start, look in **index.html** for the text that says `25 years of experience` — it should be near line 20. Put an anchor tag (`<a>`) around this text.

2. Add a `href` attribute to this anchor tag, and give it the value `/resume.html`. This URL is called a *relative path* and it looks a little different from the full URLs you're used to seeing. That's because it's actually a link to another page on the same website, so we can omit the first part. Instead of `www.jetsetter.com/resume.html` we can abbreviate the link to just `/resume.html`. Open the hint if you need a reminder on how to set attributes for html tags.

3. Click on the link that you just made to navigate to see Alejandra's resume! Click **Run** one more time when you are ready to move on to the next lesson.

```
<!DOCTYPE html>

<html>

<head>

  <title>Jetsetter Concierge Travel Planning</title>

  <link
href="https://fonts.googleapis.com/css?family=Open+Sans:300,400,
600,700|Quicksand:300,700" rel="stylesheet">

</head>

<body>

  <div class="overlay">
```

```
<div class="header">

  <div id="header-text">

    <h1>Jetsetter Concierge</h1>

    <h2>Whether you're looking for adventure or
luxury, let us help you plan your perfect getaway.</h2>

  </div>

</div>

</div>

<div class="partner">

  <h4>Proud Partner of Am South Airlines </h4>

</div>

<div class="main">

  <h3>Why You Should Choose Jetsetter Concierge Travel
Planning</h3>

  <p>With over <a href="/resume.html">25 years of
experience</a> in concierge, high-end travel planning, we'll
provide you with the highest quality services. Every vacation is
unique, custom, and tailored to your tastes.</p>

  <div class="value-props">
```

```
<div class="prop">

    <h2>Unique</h2>

    <p>We don't do standardized packages because we know everyone wants something different. Every vacation will be uniquely customized to your tastes.</p>

</div>

<div class="prop">

    <h2>Experienced</h2>

    <p>Our quarter century of experience in travel planning combines local knowledge with international taste. You'll be amazed at what we can provide!</p>

</div>

<div class="prop">
```

```
          
  
        <h2>Luxury</h2>  
  
        <p>We understand that you expect the world of  
your vacation - Jetsetter can provide once in a lifetime  
experiences at top of the line quality.</p>  
  
    </div>  
  
</div>  
  
</div>  
</body>  
</html>
```

What is CSS?

Alejandra was able to build a website that contains all of the content that she wants. But it doesn't look very impressive—Alejandra knows that a basic black and white website won't give her business the credibility she needs to find new customers.

That's where CSS comes in! CSS is the language that provides style to the content of an HTML page. This includes colors, fonts, positioning, layout, and more!

Why do we need a separate language for content and presentation? This is an example of the computer science principle *separation of concerns*. Large codebases are kept maintainable when each section has clearly differentiated problems that it is trying to solve.

Since the styling rules are described separately from the content itself, if Alejandra adds more paragraphs, images, or other forms of content, she can expect them to be styled the same way as her existing content. This will save Alejandra time in the long run, especially as her website gets more and more complex.

Applying CSS

Navigate to the **style.css** file to take a look at the CSS code that she's written.

In this file, you will recognize many of the HTML tags that you have been working with — and maybe a few that you haven't seen yet! CSS contains *selectors* that specify the HTML elements to which the style should be applied as well as *visual rules* that specify how that content should be displayed.

Now it's time to use the HTML link element to apply the CSS file to her existing website. This is done with an HTML link tag. An HTML link tag is often used to create a connection between an HTML file and the CSS file and tells the browser to apply the CSS styles to the HTML.

1. On line 6 of **index.html**, copy and paste the code below to apply the CSS file to the existing HTML. Use your knowledge of HTML attributes to figure out what this code specifies, and check out extra information in the hint if you want to learn more.

```
<link rel="stylesheet" href="style.css">
```

2.Awesome! Alejandra's site is looking good! Just a little CSS can make the difference between a website that looks like a skeleton and portfolio-ready site.Now let's tweak the CSS just a little bit, to see how the visual rules specified in CSS can change the way that a website is displayed.Navigate to the **style.css** and scroll down to line 37 where you will see the `h1` selector. This section defines the styles for the `h1` element on the page, which is the most prominent heading.Just for practice, change the `color` property from `white` to `black` and press Run to see what changes!

3.Good job changing the color of the `h1` text!!If you look at the rendered browser, you'll notice that the heading text is no longer readable in-front of the image.Let's change the color property within the `h1` selector back to white.

```
body{
    background-color: #FFFFFF;
    margin: 0px;
    padding:0px;
}

.header{
    background-image:
url(https://content.codecademy.com/programs/code-foundation
s-path/airplane.jpeg);
    background-position: center center;
    background-size: cover;
    height: 550px;
    width: 100%;
}

.overlay:before {
    content: " ";
```

```
    z-index: 1;

    display: block;

    position: absolute;

    height: 100%;

    width: 100%;

    background: rgba(0, 0, 0, 0.6);
}

.overlay * {

    position: relative; /* Makes elements display above
overlay. */

}

#header-text{

    margin: 0 auto;

    position: relative;

    text-align: center;

    top: 25%;

    width: 60%;

    z-index: 2 !important;

    font-weight: bold;

}

h1{

    color: white;

    font-family: 'Quicksand', sans-serif;

    font-size: 70px;

    font-weight: 400;
```

```
    line-height: 60px;
    margin: 0px;
}

h2{
    color: #ffffff;
    font-family: 'Open Sans', sans-serif;
    text-align: center;
    font-size: 28px;
    font-weight: 100;
}

h3{
    color: #ffffff;
    font-family: 'Quicksand', sans-serif;
    font-size: 28px;
    font-weight: 400;
    line-height: 30px;
    margin: 10px 0px;
    text-align: center;
}

h4{
    color: #cccccc;
    font-family: 'Quicksand', sans-serif;
    font-size: 16px;
    font-weight: 300;
}

p{
```

```
    color: white;
    font-family: 'Open Sans', sans-serif;
    font-size: 16px;
    font-weight: 100;
    line-height: 24px;
    margin: 0 auto;
    text-align: center;
    width: 60%;
}

.partner{
    padding: 20px;
    text-align: center;
    margin: auto;
    border-bottom: 1px solid #eeeeee;
    position: relative;
    background-color: white;
    z-index: 3;
}

.partner img{
    position: relative;
    top: 6px;
}

.aa-logo{
    padding-left: 6px;
}

.main{
```

```
        background:
url(https://content.codecademy.com/programs/code-foundation
s-path/shards.png);

        margin: 0px;

        padding: 40px;
    }

.value-props{

    margin: 40px auto;

    width: 60%;

}

.prop{

    border: 1px solid #ffffff;

    padding: 40px 0px;

    margin: 5px 0px;

    text-align: center;

}

.prop h2{

    font-family: 'Quicksand', sans-serif;

    font-size: 24px;

}

.prop li{

    color: white;

    font-family: 'Open Sans', sans-serif;

    font-size: 16px;

    font-weight: 100;

    line-height: 24px;
```

```

    margin: 0 auto;

    text-align: left;

    width: 60%;

}

<!DOCTYPE html>

<html>

<head>

    <title>Jetsetter Concierge Travel Planning</title>

    <link

href="https://fonts.googleapis.com/css?family=Open+Sans:300
,400,600,700|Quicksand:300,700" rel="stylesheet">

    <link rel="stylesheet" href="style.css">


</head>

<body>

    <div class="overlay">

        <div class="header">

            <div id="header-text">

                <h1>Jetsetter Concierge</h1>

                <h2>Whether you're looking for adventure or
luxury, let us help you plan your perfect getaway.</h2>

            </div>

        </div>

    </div>

    <div class="partner">

        <h4>Proud Partner of Am South Airlines </h4>

```

```
</div>

<div class="main">
    <h3>Why You Should Choose Jetsetter Concierge
Travel Planning</h3>

    <p>With over <a href="/resume.html">25 years of
experience</a> in concierge, high-end travel planning,
we'll provide you with the highest quality services. Every
vacation is unique, custom, and tailored to your
tastes.</p>

    <div class="value-props">
        <div class="prop">
            
            <h2>Unique</h2>
            <p>We don't do standardized packages
because we know everyone wants something different. Every
vacation will be uniquely customized to your tastes.</p>
        </div>

        <div class="prop">
            
            <h2>Experienced</h2>
            <p>Our quarter century of experience in
travel planning combines local knowledge with international
taste. You'll be amazed at what we can provide!</p>
```

```
</div>

<div class="prop">
  
  <h2>Luxury</h2>
  <p>We understand that you expect the world
of your vacation - Jetsetter can provide once in a lifetime
experiences at top of the line quality.</p>
</div>
</div>
</div>
</body>
</html>
```

What is JavaScript?

Alejandra is going to need to learn JavaScript. Any website that provides more than just static information probably utilizes JavaScript in some way. Here are some familiar examples of website features most often built with JavaScript:

popup ads, animated graphics (2D or 3D), interactive audio and video, maps and other features access the user's geographic location and much more!

One of the defining features of JavaScript is its ability to respond to browser events. These events happen in real time and can be triggered by a wide variety of user interactions, including:

the user clicking on a button, the user pressing enter on their keyboard, a video file finishing loading, the user re-sizing their window, the user hovering over text on the webpage

1. One of the classic browser events that JavaScript can respond to is the position of the mouse. When a user puts the mouse near an HTML element, the `onmouseover` event is fired. Hover over the text in the browser to see how it responds to the `onmouseover` event. Once you have observed the effect of `onmouseover`, click the **Run** button to move on to the next Checkpoint.

2. What else is happening here? JavaScript uses functions, which are reusable blocks of code designed to perform a specific task. `drawText` is a function that is being run on line 2. Change the text that says `'hello world'` to `'JavaScript'` to pass a different value into this function. Check out the browser to see what changed! One of the benefits of functions is that they are *reusable*. The `drawText` function allows us to take advantage of the reusability of functions because it can render any input text without us needing to rewrite the logic of the function. Confused? Don't worry, we'll cover functions more thoroughly in the next exercise.

```
// Change the text between the quotation marks and run your code
to see the name of your favorite language animated!

drawText('JavaScript');

// The code below will allow you to interact with the letters
bounceBubbles();
```

JavaScript Functions

In the last exercise, you started to think about two key building blocks of JavaScript: functions and events. Let's review:

Functions allow us to write a chunk of code once that can be reused over and over.

Events allow JavaScript to respond to user behaviors, like the user hovering their mouse over an HTML element or resizing their window.

Events and functions combine to give JavaScript the ability to create interactive experiences. When an event is fired, a function is executed. This pattern is used again and again in web development to create interactive websites.

Instructions

1. First, let's take a look at the JavaScript in the **index.html** file. The JavaScript code is between the `<script>` tags. This tag alerts the browser that the page contains JavaScript and separates the JavaScript code from the HTML. Find the JavaScript function `changeColor()`. This function chooses a color from a list of colors. How does that happen? First, the `getRandomColor()` function generates a random series of numbers and letters to create a new [hexadecimal color code](#). It then saves the new color under the name `newColor`.

```
let newColor = getRandomColor();
```

On the next line, the `document` keyword accesses the page's background color and sets it equal to the new color.

```
document.body.style.backgroundColor = newColor;
```

Click the **Run** button to move on to the next checkpoint to see how we can let a user change the color!

2.The next step is to connect the `changeColor()` JavaScript function with an event, so that when we click the `Repaint!` button, it changes the background color.An event is something that can happen in a browser, like clicking or hovering with your mouse. HTML attributes can set events, where the **name** of the attribute is the event and the **value** of the attribute is the JavaScript function that we want to execute.In this case, the name of the event is `onclick` and we can invoke the function by calling `"changeColor()"`. Paste this code into the opening tag of the button element: `onclick="changeColor()"`.

3.Let's test out the functionality of the code that we just added. When you click the "Repaint!" button, the background color of the site should be reset at random!When you're ready, click **Run** again and move on to the next exercise.

```
<html>

<head>

  <link rel="stylesheet" type="text/css" href="style.css">

</head>

<body>

  <button onclick="changeColor()">Repaint!</button>

</body>

<script>

  function getRandomColor() {

    let letters = '0123456789ABCDEF';

    let color = '#';

    for (let i = 0; i < 6; i++) {

      color += letters[Math.floor(Math.random() * 16)];

    }

  }

}
```

```
    }

    return color;

}

function changeColor() {

    let newColor = getRandomColor();

    document.body.style.backgroundColor = newColor;

}

</script>

</html>
```

What is SQL?

SQL stores information in tables, which is simply a collection of information organized into rows and columns. If you've worked with a spreadsheet like Microsoft Excel or Google Sheets, you might be familiar with working with tables. To get the information, you would write a *query*, or a program that would retrieve data from the table.

Here's an example of a query that would select all of the columns —the `*` keyword is a shorthand for “all”— from the `page_views` table:

```
SELECT * FROM
```

Web developers and software engineers also use SQL to build apps that can save, modify, and access data. There's even a growing field of *data engineering*,

which is a specialized subset of software engineers who ensure that the applications they are working with accurately and efficiently record all of the required data.

1.

1. Now you try it! Add `SELECT * FROM users;` to the text editor to select the entire `users` table from the database. Press run and take a look at the data that is returned. What kinds of data is Alejandra's app able to store and retrieve? Why is it important for her to store this particular set of information?

```
SELECT * FROM users;
```